Iterator

Season: 3 · Behavioral · Category: GoF · Level: Intermediate · Build with Waqar

Today's Hook

- Today: Iterator as a StreamForge product force — not trivia.
- Runnable Demo on JDK 25 with captured stdout.
- Classic encoding → Java 25 / Zazr modernization → trade-offs.
- CapCut slides + full spoken script included.

The Problem

- Naive local fixes: more ifs, statics, nulls, or threads.
- Illegal states still compile; tests become order-dependent.
- Design reviews lack a shared name for the force.
- Interviews punish memorization without judgment.

Structure



Iterator structure

Real-World Scenario

- StreamForge creator platform is the running metaphor.
- Same names appear in naive code, refactor, and Demo output.
- Stay in one story for the full 10–12 minutes.
- Product language first; pattern vocabulary second.

Naive Code

```
// Naive sketch for Iterator
public class Naive {
  // duplicated logic / leaked details / unsafe sharing
  Object doIt(Object... args) {
    if (args == null) return null;
    return args[0];
  }
}
```

Why Naive Hurts

- Coupling spreads across controllers and infrastructure.
- Tests need sleeps or static reset hooks.
- New variants require shotgun edits.
- On-call cannot reason about failure modes.

Implementation

```java
// file: src/main/java/com/buildwithwaqar/patterns/gof/behavioral/iterator/RoundRobin.java
package com.buildwithwaqar.patterns.gof.behavioral.iterator;

import java.util.Iterator;
import java.util.List;
import java.util.NoSuchElementException;

/** Custom iterator that cycles through a non-empty list. */
public final class RoundRobin<T> implements Iterable<T> {
    private final List<T> items;

    public RoundRobin(List<T> items) {
        if (items.isEmpty()) throw new IllegalArgumentException("empty");
        this.items = List.copyOf(items);
    }

    @Override
    public Iterator<T> iterator() {
        return new Iterator<>() {
            int i = 0;
            public boolean hasNext() { return true; }
            public T next() {
                if (items.isEmpty()) throw new NoSuchElementException();
                var v = items.get(i % items.size());
                i++;
                return v;
            }
```

Implementation (2)

```
        };
    }
}
```

Demo.main

```java
package com.buildwithwaqar.patterns.gof.behavioral.iterator;
import java.util.List;
public final class Demo {
  public static void main(String[] a) {
    System.out.println("=== Iterator walkthrough ===");
    var it = new RoundRobin<>(List.of("worker-a", "worker-b", "worker-c")).iterator();
    for (int i = 0; i < 5; i++) System.out.println("next=" + it.next());
  }
}
```

Walkthrough Output

```
=== Iterator walkthrough ===
next=worker-a
next=worker-b
next=worker-c
next=worker-a
next=worker-b
```

Step-by-Step Trace

- Each stdout line maps to a refactor stage.
- Re-run Demo locally on JDK 25 to verify.
- Differences usually mean a dirty tree — pull main.
- Pause the video and predict the next line before revealing.

Modern Java 25 / Zazr

```
// Java 25 / Zazr direction for Iterator
// records + sealed + switch
// Option/Either/Validation instead of null
// virtual threads for blocking fan-out
// inject interfaces at the edge
```

Trade-offs

- Clarity vs ceremony — do not spawn five types for two lines.
- Closed sealed worlds vs open SPI plugins.
- Teaching demos omit production idempotency/metrics.
- Prefer simpler language features when they erase the force.

Related Patterns

- Contrast with the neighbor pattern (force difference).
- Name the roles from memory in sixty seconds.
- Point to the season map for what comes next.
- Link the GitHub episode folder in the description.

Recap

- Force → naive pain → Iterator → Demo stdout.
- Modernize with Java 25 / Zazr when it shrinks code.
- Teach when NOT to use it.
- Practice: break a test, fix it, re-run Demo.